RI.UVT.RO Redesign
Developer Sync Guide

Direcția de Relații Internaționale
West University of Timișoara

Focus: A practical synchronization guide for React and WordPress developers working across role folders and the main repository.

Role 5 + Role 6 synchronization instructions

Technology Stack: WordPress CMS + REST API + React + Vite + TypeScript
Team Q-UW9

Contents

Document standard: This file follows the RI.UVT.RO document styling system: institutional UVT blue hierarchy, yellow accent rules, readable spacing, structured tables, and technical code blocks.

1. Developer Sync Guide

Written: 2026-05-12
Applies to: Role 5 (React Developer) and Role 6 (WordPress Developer)

1.1 Overview

Each role works in their own folder under `roles/roleX/ro.uvt.ri/`. The main branch lives at `ro.uvt.ri/`. Changes move in one direction at a time:

```
main (ro.uvt.ri/)
  ^  merge approved changes in
  v  pull updates out

roles/role5/ro.uvt.ri/    roles/role6/ro.uvt.ri/
```

- You never edit files directly in `ro.uvt.ri/` (main). You work in your role folder and submit your changes for review.
- You pull from main into your role folder regularly so you are always building on the current agreed state.
- Docs inside your role folder (`roles/roleX/ro.uvt.ri/docs/`) are your working copies only. They are not reviewed or merged. The canonical docs are in `ro.uvt.ri/docs/`.

1.2 How to Pull Updates from Main into Your Role Folder

When main is updated (after a merge session like this one), you need to bring those changes into your local working copy.

1.2.1 Frontend changes

Copy the changed files or folders from `ro.uvt.ri/frontend/src/` into your role's `frontend/src/`:

```
ro.uvt.ri/frontend/src/
  -> roles/roleX/ro.uvt.ri/frontend/src/
```

Do this selectively – only copy what actually changed, not the entire folder each time. Check `14_Week3_End_Changes.md` (and future change reports) to see exactly which files were updated.

After the Week 3 merge, the files that changed in main are:

File	What changed
`frontend/src/App.tsx`	`/test` route added for `TestPage`
`frontend/src/pages/TestPage.tsx`	Import path fixed (`molechules` → `molecules`)

Copy those two files from main into your role folder. Everything else in `frontend/src/` is already the same.

1.2.2 WordPress changes (Role 6 only)

Two plugin folders were added to main's `wordpress/ro.uvt.ri/wp-content/plugins/`:

```
wordpress/ro.uvt.ri/wp-content/plugins/advanced-custom-fields/
wordpress/ro.uvt.ri/wp-content/plugins/custom-post-type-ui/
```

Role 6: if you already have these installed in your Laragon setup, no action is needed on your machine. Just make sure your `roles/role6/ro.uvt.ri/wp-content/plugins/` folder reflects this too.

1.2.3 `.gitignore` change (all roles)

The `.gitignore` in main changed. Copy it into your role folder:

```
ro.uvt.ri/.gitignore  ->  roles/roleX/ro.uvt.ri/.gitignore
```

This ensures that when you look at what git would track in your role folder, it matches the same rules as main.

1.3 How to Submit Your Changes to Main

When you have finished your Week 4 / Week 5 work and it is ready to be reviewed, follow these steps.

1.3.1 Step 1 – Know what you changed

Before you submit anything, make a clear list of: - Which files you added - Which files you modified - What each change does

Do not submit files you did not touch. Do not submit your `docs/` folder (it is not reviewed).

1.3.2 Step 2 – Check against main's current state

Open the equivalent file in `ro.uvt.ri/` and compare it with your version. If main has a newer version of a file that you also modified, you need to reconcile the two manually before submitting – do not just overwrite main's version with yours.

Common conflict areas: - `App.tsx` – both roles may add routes at the same time - `frontend/src/api/wordp` – Role 5 expands the API layer; do not submit an older version of this file

1.3.3 Step 3 – Copy your changes into main

Copy only the files you worked on from your role folder into the corresponding location in `ro.uvt.ri/`.

Example for Role 5:

```
roles/role5/ro.uvt.ri/frontend/src/components/organisms/Navbar/Navbar.tsx
  -> ro.uvt.ri/frontend/src/components/organisms/Navbar/Navbar.tsx
```

Example for Role 6:

```
roles/role6/ro.uvt.ri/wp-content/plugins/my-custom-plugin/
  -> ro.uvt.ri/wordpress/ro.uvt.ri/wp-content/plugins/my-custom-plugin/
```

1.3.4 Step 4 – Update the change report

Add your changes to the current week's change report in `ro.uvt.ri/docs/agent_memory/`. If the file for the current week does not exist yet, create one following the naming pattern (`15_`, `16_`, etc.).

1.4 Role 5 – What to Work On Next (Week 4 / Week 5)

Your component library is built and in main. The next phase is wiring it to real routes and real data.

1.4.1 Immediate priorities

1. **Expand the API layer** – `frontend/src/api/wordpress.js` currently only has `getPosts` and `getPost`. You need to add helpers for every CPT once Role 6 registers them:
   - `getProgrammes()`
   - `getResources()`
   - `getCalls()`
   - `getStories()`

2. **Build the route structure** – `App.tsx` has only two routes (`/` and `/test`). You need to add the full route tree from `01_Sitemap.md`. Start with the top-level routes, then add nested ones:

```
/erasmus
/erasmus/incoming-students
/erasmus/outgoing-students
/international-students
/programmes
/calls
/stories
/resources
/contact
```

3. **Build page templates** – For each route, create a page file in `frontend/src/pages/`. Use the existing organisms (`Navbar`, `Footer`, `HeroSection`, `AccordionSection`) and templates (`PageLayout`, `ContentGrid`).

4. **Connect components to API data** – Replace hardcoded content with data fetched from the WordPress REST API. Refer to `07_API_Contract.md` for endpoint specs.

1.4.2 Rules to follow while working

- Never fetch data directly inside a component. All API calls go through `frontend/src/api/wordpress.j`
- Keep `App.tsx` clean – one import and one `<Route>` per page.
- Build page files in `frontend/src/pages/`, not inside components.
- Do not create new atoms or molecules without first checking that the needed one does not already exist in `components/atoms/` or `components/molecules/`.

1.4.3 File structure reminder

```
frontend/src/
  api/
    wordpress.js          <- all API calls go here
  components/
    atoms/                <- Button, Typography
    molecules/            <- Card, SectionHeader
    organisms/            <- Navbar, Footer, HeroSection, AccordionSection,
    ↪  TabsSection
    templates/            <- PageLayout, ContentGrid
  layouts/
    Container.tsx
  pages/                  <- one file per route
  styles/
    globals.css
  App.tsx                 <- route declarations only
```

`main.tsx`

1.5 Role 6 – What to Work On Next (Week 4 / Week 5)

The plugins are now in the repo and visible to git. The next phase is configuring the WordPress backend so the REST API can serve real content to the frontend.

1.5.1 Immediate priorities

1. **Activate plugins in WordPress admin**
   - Open Laragon, start Apache and MySQL
   - Navigate to `http://ro.uvt.ri.test/wp-admin`
   - Go to Plugins -> activate **Advanced Custom Fields** and **Custom Post Type UI**
2. **Register Custom Post Types using CPTUI**

   Create the following CPTs as defined in `06_WordPress_Content_Model.md`:

CPT slug	Label	Has archive
`call`	Calls	Yes
`story`	Stories	Yes
`resource`	Resources	Yes
`programme`	Programmes	Yes
`people`	People	Optional – confirm with Role 1

   For each CPT, enable: **REST API support** (show_in_rest = true). This is required for the frontend to consume it.
3. **Register Taxonomies using CPTUI**

Taxonomy slug	Label	Attached to
`audience`	Audience	All CPTs
`programme-family`	Programme Family	`programme`
`content-topic`	Content Topic	`resource`, `story`
`academic-year`	Academic Year	`call`, `resource`

4. **Create ACF field groups** for each CPT. At minimum create the following fields per CPT so the frontend has structured data to consume:
   - **Calls:** deadline (date), eligibility (textarea), application steps (repeater), documents (repeater: label + file URL)

- **Programmes:** duration, language, partner institution, application deadline
- **Resources:** file URL, file type, audience notes
- **Stories:** author, date, pull quote

5. **Create test entries** – Add at least one published entry for each CPT so Role 5 can test their API helpers against real data.
6. **Verify REST API responses** – For each CPT, check that the endpoint returns data:

   ```
   http://ro.uvt.ri.test/wp-json/wp/v2/calls
   http://ro.uvt.ri.test/wp-json/wp/v2/stories
   http://ro.uvt.ri.test/wp-json/wp/v2/resources
   http://ro.uvt.ri.test/wp-json/wp/v2/programmes
   ```

   The ACF field values must appear in the REST response. If they do not, enable REST API support per field group inside ACF settings.

1.5.2 What to submit at end of Week 5

Your deliverable is not code – it is a configured WordPress state. Since database content cannot be committed to git, your submission should include:

- Any **custom plugin files** you write (e.g., a functions plugin that registers CPTs in code rather than via CPTUI UI)
- A **documentation file** at `roles/role6/ro.uvt.ri/docs/week5/` describing:
  - Which CPTs were registered and with what settings
  - Which field groups were created and what fields they contain
  - Sample REST API response for at least one CPT
  - Any issues or deviations from the original content model

1.5.3 Fixing your folder structure

Your role folder has WordPress files at the wrong level. Currently:

```
roles/role6/ro.uvt.ri/
  wp-admin/          <- wrong -- WP core at root
  wp-content/
  wp-includes/
  frontend/
  docs/
```

It should match main's structure:

```
roles/role6/ro.uvt.ri/
  wordpress/
    ro.uvt.ri/
```

```
    wp-content/   <- WP lives here
frontend/
docs/
```

Move your WP files into `wordpress/ro.uvt.ri/` before your next submission. This does not affect your Laragon setup – Laragon reads from `C:/laragon/www/`, not from the repo folder.

1.6 Shared Rules (Both Roles)

Rule	Reason
Never edit files directly in `ro.uvt.ri/` while also working in your role folder	You will overwrite your own work on the next sync
Always check the latest change report before starting a new work session	Main may have changed since you last synced
Only submit files you authored or intentionally modified	Submitting unchanged files creates false merge conflicts
Do not submit your `docs/agent_memory/` folder	The canonical agent memory lives in main only
Do not submit `node_modules/`, `dist/`, `.env`, or `wp-content/uploads/`	These are gitignored for good reason
Keep your local Laragon DB in sync with any CPT or field group changes Role 6 documents	The DB is not tracked – communication is the only sync mechanism